RES POLYTECHNIC, BANGALORE-91

CIE Theory Test III - Model Answers

Data Structure with Python | Course Code 25CS33I | Marks 50 | Semester III

SECTION I

Q1 a) Tree, terms and properties (5 marks)

Tree: a non-linear hierarchical data structure made of nodes connected by edges, with one topmost node called the root.

For the given tree (50 at the top; 30, 70 below it; 20, 40, 60, 80 at the bottom):

Term	Answer
Root node	50
Leaf nodes	20, 40, 60, 80
Parent of node 40	30
Children of node 30	20 and 40
Height of tree	2 (counted in edges; 3 levels)

Properties (any two):

- A tree with n nodes has exactly n – 1 edges.
- There is exactly one path between any two nodes (no cycles).
- Maximum nodes at level L of a binary tree = 2^L.

Q1 b) Recursion, factorial, recursion vs iteration (10 marks)

How recursion works: a function calls itself. Each call is pushed as a new frame onto the call stack (memory for its parameters, local variables and return address). Calls keep stacking until the **base case** is reached; then the frames are popped one by one (LIFO) and each returns its value to the caller.

```
def fact(n):
    if n == 0 or n == 1:        # base case
        return 1
    return n * fact(n - 1)      # recursive case

print(fact(4))                  # 24
```

Trace for n = 4:

```
fact(4) = 4 * fact(3)
fact(3) = 3 * fact(2)
fact(2) = 2 * fact(1)
fact(1) = 1                  <- base case, stops
Return:  fact(2) = 2*1 = 2
         fact(3) = 3*2 = 6
         fact(4) = 4*6 = 24
```

Recursion	Iteration
Function calls itself	Uses loops (for / while)
Needs base case to stop	Needs loop condition to stop
Uses stack memory for every call (more memory)	Uses fixed memory
Code is shorter but slower	Code is longer but faster
Risk of stack overflow	No such risk

Q1 c) Division with exception handling (10 marks)

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print("Result =", a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input - enter numbers only")
finally:
    print("Program completed")
```

Output (10 and 0): `Cannot divide by zero` / `Program completed`

- `try` holds risky code; `except` handles a particular error; `finally` always runs (whether error occurs or not).

OR

Q2 a) Traversals of the tree (5 marks)

Tree: root 40; left child 20 (children 10, 30); right child 60 (right child 70).

- **In-order (Left, Root, Right):** 10, 20, 30, 40, 60, 70
- **Pre-order (Root, Left, Right):** 40, 20, 10, 30, 60, 70
- **Post-order (Left, Right, Root):** 10, 30, 20, 70, 60, 40

Q2 b) Types of errors; element at index with exception handling (10 marks)

Types of errors:

1. **Syntax error** – wrong grammar, found before running (e.g. missing colon).
2. **Runtime error (exception)** – occurs while running (e.g. ZeroDivisionError, IndexError).
3. **Logical error** – program runs but gives wrong output (e.g. using + instead of *).

```
try:
    nums = list(map(int, input("Enter numbers separated by space: ").split()))
    index = int(input("Enter index: "))
    print("Element =", nums[index])
except IndexError:
    print("Index is out of range")
except ValueError:
    print("Invalid input - enter integers only")
```

Output (list 10 20 30, index 5): `Index is out of range`

Q2 c) ADT, UDT and Stack class (10 marks)

- **ADT (Abstract Data Type):** a model that defines *what* operations a data structure supports, not *how* they are implemented (e.g. Stack with push, pop).
- **UDT (User-Defined Data Type):** a type created by the programmer using a class to implement an ADT.
- **Four common ADTs:** Stack, Queue, List, Dictionary (also Set, Tree, Graph).

```
class Stack:                        # UDT implementing Stack ADT
    def __init__(self):
        self.items = []

    def push(self, x):
        self.items.append(x)

    def pop(self):
        if self.items == []:
            return "Stack Underflow"
        return self.items.pop()

s = Stack()
s.push(10); s.push(20)
print(s.pop())       # 20
print(s.pop())       # 10
print(s.pop())       # Stack Underflow
```

SECTION II

Q3 a) Output of the Stack code (10 marks)

Trace: push 10, 20, 30 → [10, 20, 30]. pop → 30. peek → 20. push 40 → [10, 20, 40]. pop → 40, pop → 20, pop → 10. is_empty → True. pop on empty → "Stack Underflow".

```
30
20
40
20
10
True
Stack Underflow
```

Q3 b) Output of the Linked List code (15 marks)

Trace:

1. insert_end(10), insert_end(20) → 10 → 20; insert_begin(5) → 5 → 10 → 20. **display:** `5 -> 10 -> 20 -> None`
2. insert_end(30) → 5, 10, 20, 30. delete_value(20) removes 20 → 5, 10, 30. **display:** `5 -> 10 -> 30 -> None`

3. delete_value(99): not in list → prints `99 not found`.
4. delete_value(5): head node deleted → list is 10, 30. **display:** `10 -> 30 -> None`

```
5 -> 10 -> 20 -> None
5 -> 10 -> 30 -> None
99 not found
10 -> 30 -> None
```

OR

Q4 a) Queue using a class (12 marks)

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)
        print(item, "added to queue")

    def dequeue(self):
        if self.is_empty():
            print("Queue is empty")
        else:
            print(self.items.pop(0), "removed from queue")

    def is_empty(self):
        return len(self.items) == 0

    def display(self):
        if self.is_empty():
            print("Queue is empty")
        else:
            print("Queue:", self.items)

q = Queue()
for name in ["Asha", "Ravi", "Meena", "Kiran"]:
    q.enqueue(name)
q.dequeue()
q.dequeue()
q.display()
```

Output: `Asha removed`, `Ravi removed`, `Queue: ['Meena', 'Kiran']`

Suitable Python concept: Class and Object (OOP) with a list. A queue is a FIFO ADT; a class bundles the data (list) and the operations (enqueue, dequeue) together, hides the details, and allows many queue objects to be created. A list's `append()` adds at the rear and `pop(0)` removes from the front, which matches FIFO. (`collections.deque` can be used for faster removal.)

Q4 b) Doubly Linked List (13 marks)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        if self.head:
            self.head.prev = new
        self.head = new

    def delete_value(self, key):
```

```
        temp = self.head
        while temp and temp.data != key:
            temp = temp.next
        if temp is None:
            print(key, "not found")
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev

    def display(self):
        temp, last = self.head, None
        print("Forward :", end=" ")
        while temp:
            print(temp.data, end=" <-> ")
            last = temp
            temp = temp.next
        print("None")
        print("Backward:", end=" ")
        while last:
            print(last.data, end=" <-> ")
            last = last.prev
        print("None")

dll = DoublyLinkedList()
dll.insert_end(10); dll.insert_end(20); dll.insert_end(30)
dll.insert_begin(5)
dll.delete_value(20)
dll.display()
```

Output:

```
Forward : 5 <-> 10 <-> 30 <-> None
Backward: 30 <-> 10 <-> 5 <-> None
```

Suitable Python concept: Classes with object references (linked structure). Each Node object stores data plus `prev` and `next` references, so nodes can be anywhere in memory and the list grows/shrinks dynamically. Insertion/deletion only needs pointer changes (no shifting as in a list), and the `prev` pointer allows backward traversal.